

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

Отчет по блоку задач №2 (Яндекс.Контест)

Выполнил:

Студент группы Р3209

Евграфов А. А.

Преподаватель:

Косяков М. С.

Санкт-Петербург, 2026 г.

Задачи из 2-го блока Яндекс.Контеста

Задача №5 «Е. Коровы в стойла»

Описание

На прямой расположены N стойл с известными координатами (возрастающая последовательность). Требуется расставить K коров так, чтобы минимальное расстояние между любыми двумя коровами было максимально возможным. Вывести это максимальное минимальное расстояние.

Ограничение времени: 0.1 секунда.

Ограничение памяти: 256 Мб.

Решение

Сортируем стойла по координатам. Бинарным поиском по ответу находим максимально допустимое расстояние между коровами. Заметим, что разместить коров подряд можно всегда, а на расстояние равную длине отрезка между первым и последним стойлом - нет. Первую корову размещаем левее всего (потому что это всегда выгодно), остальных строго на полученное расстояние.

Доказательство

Пусть стойла отсортированы по возрастанию. Заметим, что множество минимальных расстояний между стойлами имеет свойство монотонности: если коров можно разместить с минимальным расстоянием x , то и с минимальным $k < x$ тоже можно. Поддерживем инвариант: коров можно разместить с минимальным расстоянием 1, а при расстоянии g - нельзя (базовые значения корректны). Тогда, допустив, что чекер решения работает корректно, бинарный поиск корректно найдёт такое минимальное $x_0 = l + 1$, что минимальное расстояние x_0 не является валидным, значит ответ - l .

Докажем, что чекер решения корректен. Пусть x - кандидат на ответ. Заметим, что расположение коров с минимальным расстоянием x валидно тогда и только тогда, когда коров можно разместить следующим образом: первую корову в первое стойло, i -ую корову на расстояние x от $(i - 1)$ -ой. Действительно: если между коровами в нашей оптимальной расстановке увеличить расстояние между какими-то двумя, то следующие коровы после правой передвинутой не могут расположиться левее чем были - стойл может не хватить. А располагать левую корову в самой первой стойле выгодно всегда (уж совсем очевидно).

Алгоритмическая сложность

- По времени: $O(n \log(n))$
- По памяти: $O(1)$

Код

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 bool check(const vector<int>& arr, int k, int x) {
6     int count = 1;
7     int last = arr[0];
8     for (size_t i = 1; i < arr.size(); i++) {
9         if (arr[i] - last >= x) {
10             last = arr[i];
11             count++;
12         }
13     }
14     return count >= k;
15 }
16
17 int main() {
18     ios_base::sync_with_stdio(false);
19     cin.tie(nullptr);
20
21     int n;
22     int k;
23     cin >> n >> k;
24
25     vector<int> arr(n);
26     for (int i = 0; i < n; i++) {
27         cin >> arr[i];
28     }
29
30     sort(arr.begin(), arr.end());
31
32     int l = 0;
33     int r = arr[n - 1] - arr[0] + 1;
34     int m;
35
36     while (l + 1 < r) {
37         m = l + (r - l) / 2;
38         if (check(arr, k, m)) {
39             l = m;
40         } else {
41             r = m;
42         }
43     }
44
45     cout << l;
46     return 0;
47 }

```

Задача №6 «Г. Максимальное число»

Описание

Даны несколько фрагментов числа (строки цифр). Требуется составить из них максимально возможное число, соединяя фрагменты в любом порядке. Каждая строка содержит от 1 до 100 цифр, общее количество строк не превышает 100. Гарантируется, что хотя бы одна строка начинается не с нуля, поэтому итоговое число не будет иметь ведущих нулей, за исключением случая, когда все строки состоят из нулей – тогда ответ 0.

Ограничение времени: 1 секунда.

Ограничение памяти: 256 Мб.

Решение

Заметим, что если число, образованное склейкой фрагментов a и b (именно в таком порядке) больше склейки b и a , то a выгоднее расположить левее, чем b . Сортируем фрагменты и выводим от большего к меньшему.

Доказательство

Скажем, что фрагмент $A \succ B$, если конкатенация строк $A + B > B + A$ (как числа или лексикографически, так как длины получающихся строк равны). Введённое отношение действительно является иррефлексивным, давайте покажем транзитивность.

Пусть A и B – строки, представляющие числа. Обозначим их длины как $|A|$ и $|B|$. Условие $A \succ B$ равносильно:

$$A \cdot 10^{|B|} + B > B \cdot 10^{|A|} + A$$

$$A \cdot 10^{|B|} - A > B \cdot 10^{|A|} - B$$

$$A \cdot (10^{|B|} - 1) > B \cdot (10^{|A|} - 1)$$

Выражения в скобках положительны: $\frac{A}{10^{|A|}-1} > \frac{B}{10^{|B|}-1}$

Рассмотрим $f(X) = \frac{X}{10^{|X|}-1}$. Она действует из множества строк в $\mathbb{R}$, тогда так как $X \succ Y$ равносильно $f(A) > f(B)$, то если $A \succ B$ и $B \succ C$, то $A \succ C$ в силу транзитивности отношения порядка больше-меньше на $\mathbb{R}$.

Сортировка отработает корректно.

Предположим, существует какая-то другая перестановка фрагментов, которая дает число M_0 , строго большее, чем число, полученное нашей сортировкой. Если последовательность M_0 не отсортирована по отношению $\succ$, значит в ней есть два соседних фрагмента X и Y (сначала X затем Y), для которых $X \prec Y$.

$$M_0 = 10^l \cdot A + 10^k(10^{|Y|}X + Y) + B$$

Давайте поменяем местами эти фрагменты. Новое число M_1 будет иметь вид: $M_1 = 10^l \cdot A + 10^k(10^{|X|} \cdot Y + X) + B$.

Тогда $M_1 < M_0$ равносильно $10^{|X|} \cdot Y + X < 10^{|Y|} \cdot X + Y$, то есть $X \succ Y$ - противоречие.

Алгоритмическая сложность

- По времени: $O(n \log(n))$
- По памяти: $O(1)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 bool comp(const string& a, const string& b) {
6     return a + b > b + a;
7 }
8
9 int main() {
10     ios_base::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     vector<string> arr;
14     string s;
15
16     while (cin >> s) {
17         arr.push_back(s);
18     }
19
20     sort(arr.begin(), arr.end(), comp);
21
22     for (size_t i = 0; i < arr.size(); i++) {
23         cout << arr[i];
24     }
25
26     return 0;
27 }
```

Задача №7 «Г. Кошмар в замке»

Требуется переставить буквы в строке так, чтобы максимизировать сумму по всем буквам: вес буквы, умноженный на максимальное расстояние между позициями этой буквы в полученной строке. Если буква встречается один раз, расстояние считается равным 0. Вывести любую из строк с максимальным весом.

Ограничение времени: 2 секунды.

Ограничение памяти: 64 Мб.

Решение

Действуем жадно: выделяем те буквы, которые встречаются хотя бы два раза и сортируем их по возрастанию веса. Строим строку следующим образом: сначала две самые дорогие буквы парой, потом между этими буквами ставим самую дорогую пару из оставшихся и так пока не закончатся буквы. Остальные ставим в середину произвольным образом. То есть самые дорогие буквы будут находиться на самом большом расстоянии друг от друга.

Доказательство

Пусть $a_1 > a_2 > \dots > a_n$ - буквы, которые встречаются хотя бы два раза, отсортированные по убыванию веса. Заметим, что остальные буквы (которые встречаются 1 раз) и остальные дубликаты a_i можно не рассматривать, поскольку они не влияют на цену итоговой строки. Пусть $d_1 > d_2 > \dots > d_n$ - расстояние между буквой a_i и её парой в итоговой строке. Тогда мы хотим максимизировать величину $\sum_{i=1}^n a_{\sigma_1(i)} d_{\sigma_2(i)}$, где σ_1, σ_2 - произвольные перестановки на $\{1, 2, \dots, n\}$. Согласно трансервенству максимум достигается тогда, когда $\sigma_1(i) = \sigma_2(i) = i$, значит наш алгоритм корректен.

Алгоритмическая сложность

- По времени: $O(n)$
- По памяти: $O(n)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     string s;
10    cin >> s;
11
12    vector<int> c(26);
```

```

13  for (int i = 0; i < 26; i++) {
14      cin >> c[i];
15  }
16
17  unordered_map<char, int> mp;
18  for (char ch : s) {
19      mp[ch]++;
20  }
21
22  vector<char> v;
23  for (int i = 0; i < 26; ++i) {
24      if (mp[i + 'a'] >= 2) {
25          v.push_back(i + 'a');
26      }
27  }
28
29  sort(v.begin(), v.end(), [&](char a, char b) { return c[a - 'a'] > c[b - 'a']; });
30
31  string res = "";
32  string tail = "";
33
34  for (char ch : v) {
35      res += ch;
36      tail += ch;
37      mp[ch] -= 2;
38  }
39
40  reverse(tail.begin(), tail.end());
41
42  for (int i = 0; i < 26; ++i) {
43      while (mp[i + 'a'] > 0) {
44          res += static_cast<char>(i + 'a');
45          mp[i + 'a']--;
46      }
47  }
48
49  res += tail;
50
51  cout << res;
52
53  return 0;
54 }

```

Задача №8 «Н. Магазин»

Описание

Имеется n товаров с ценами a_i . Можно разбить их на несколько чеков (произвольное количество). В каждом чеке, если в нём m товаров, то $\lfloor \frac{m}{k} \rfloor$ самых дешёвых товаров в этом чеке достаются бесплатно. Требуется минимизировать общую сумму, которую заплатят.

Ограничение времени: 2 секунды.

Ограничение памяти: 256 Мб.

Решение

Сортируем цены, выбираем каждый k -ый товар, начиная с самого дорогого.

Доказательство

Пусть $l = \lfloor \frac{n}{k} \rfloor$. Заметим, что больше, чем l товаров получить бесплатно не получится. Пусть наш алгоритм выдал товары с ценами $a_1 > a_2 > \dots > a_l$, но существует какое-то другое решение $b_1 > b_2 > \dots > b_l$. Заметим, что $b_i \leq c_{i \cdot k}$, где $c_1 > c_2 > \dots > c_n$ - все цены, а равенство достигается тогда, когда $b_i = a_i$.

Алгоритмическая сложность

- По времени: $O(n \log(n))$
- По памяти: $O(1)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     int n;
10    int k;
11    cin >> n >> k;
12
13    vector<int> arr(n);
14    for (int i = 0; i < n; i++) {
15        cin >> arr[i];
16    }
17
18    sort(arr.begin(), arr.end());
19    reverse(arr.begin(), arr.end());
20
```



```
21 long long count = 0;
22 for (int i = 0; i < n; i++) {
23     if ((i + 1) % k != 0) {
24         count += arr[i];
25     }
26 }
27
28 cout << count;
29 return 0;
30 }
```